

Overview

For this simple Chameleon project, you will be setting up a digital soil prong powered by a BBC micro:bit. The goal of this project is to grow an assortment of microgreens and lentils. The soil prong will provide you when to water the plants you set up & will locally collect moisture, light, and temperature data for Team Chameleon to study. Once the project is over, you can then upload the file onto a Google Form which we will use to optimize our curriculum and environmental understanding.

Seed Prep

Inside the tote bag you received, there will be a small parchment paper pouch, tied with twine. Untie this pouch with caution, since there are many small seeds inside. The seeds are an assortment of lentils, and before they can be planted in dirt, they need to be soaked overnight. Place the seeds in a container of water and let it sit for 8 hours. After this, the seeds are ready to be planted.

Soil Prep

For the soil and pot setup, anything will suffice. As long as your pot is big enough to hold the soil prong, it'll support the seeds too. Soil that is most like garden dirt is optimal, and dirt with wood chips or stones should try to be avoided.

Micro:bit Setup

1. Take out battery pack and insert two batteries inside
2. Unfold cardstock battery sleeve and follow folding steps specified on the front
3. Once the battery sleeve is on, take out the micro:bit and the Kritonik soil moisture prong
4. Grab three screws and three bolts from the bag
5. Insert the screws into the corresponding holes on the micro:bit and the prong labeled P1, 3V and GND
6. Once these are secured, use the empty holes on the cardstock sleeve to secure the microbit to the battery pack
7. Grab the dangling wire from the battery pack and insert it into the large white port on the left of the micro:bit
8. Write code at makecode.microbit.org/#editor*

9. Connect the micro:bit to your computer using the included micro USB cable. Accept the request to mount to your computer. In the MakeCode editor, click the “Download” option which will walk you through the rest of the process.
10. Here is the code for the simple version:

```
reading = 0

def on_forever():
    global reading
    reading = pins.analog_read_pin(AnalogPin.P0)
    led.plot_bar_graph(reading, 1023)
    if input.button_is_pressed(Button.A):
        basic.show_number(reading)

basic.forever(on_forever)
```

11. Here is the advanced code that is capable of saving data to your micro:bit’s flash memory:

```
import datalogger

# Constants for reading sensor and battery saving
led.enable(True)
MAX_MOISTURE = 1023 # Minimum analog value (dry)
MIN_MOISTURE = 0    # Maximum analog value (wet)
TOO_DRY_THRESHOLD = 40.0 # Moisture percentage below which soil is too dry
TOO_WET_THRESHOLD = 80.0 # Moisture percentage above which soil is too wet
TEMP = 0.0

# Time constants
ONE_MINUTE_MS = 60000
MEASUREMENT_INTERVAL_MS = 15 * ONE_MINUTE_MS # 15-minute interval for readings

# Initialize datalogger with headers
datalogger.include_timestamp(FlashLogTimeStampFormat.MINUTES)
datalogger.set_columns(["temperature", "moisture"])

def read_soil_moisture():
    """
    Reads the analog value from the soil sensor and returns
    the percentage of moisture.
    """
    raw_value = pins.analog_read_pin(AnalogPin.P1) # Use pin0 directly
    moisture_percentage = 100 - ((raw_value - MAX_MOISTURE) * 100 / (MIN_MOISTURE -
MAX_MOISTURE))
```

```

    return moisture_percentage

def log_data(moisture):
    """
    Logs data using the datalogger.
    """
    datalogger.log(datalogger.create_cv("temperature", input.temperature()),
                  datalogger.create_cv("moisture", moisture))

def get_timestamp():
    """
    Returns a simple timestamp in minutes since the micro:bit was powered on.
    """
    milliseconds = input.running_time() # Correct function name
    minutes = milliseconds // 60000
    return minutes

def display_moisture_status(moisture):
    """
    Continuously displays a face based on the moisture level.
    - Custom sad face if soil is too dry (< 40%)
    - Custom happy face if moisture is optimal (40% <= moisture <= 80%)
    - Custom symbol if soil is too wet (> 80%)
    """
    if moisture < TOO_DRY_THRESHOLD:
        # Custom sad face
        basic.show_icon(IconNames.SAD)
    elif moisture > TOO_WET_THRESHOLD:
        # Custom droplet shape for too wet
        basic.show_icon(IconNames.NO)
    else:
        # Custom happy face
        basic.show_icon(IconNames.HAPPY)

def save_moisture_reading():
    """
    Reads soil moisture, logs the data with a timestamp,
    and updates the display with the current moisture status.
    """
    moisture = read_soil_moisture()
    timestamp = get_timestamp()
    TEMP = int(input.temperature())
    log_data(moisture)
    display_moisture_status(moisture)

while True:
    # Save a reading and update the display every 15 minutes

```

```
save_moisture_reading()
```

```
# Go to sleep for 15 minutes to conserve battery
```

```
basic.pause(MEASUREMENT_INTERVAL_MS) # Use sleep correctly
```

12. Download code off the makeCode platform

13. Insert your soil moisture sensor into a houseplant or some samples you gather and watch the magic happen!

(all readings from 0-1023 are in microvolts (mV) signifying the conductivity of your soil)

***Micro:bit Code**

No coding or applications need to be installed in order to program your micro:bit. Instead, microsoft makecode will be used and all data collected by your micro:bit setup will be downloaded locally.